CSEN3083

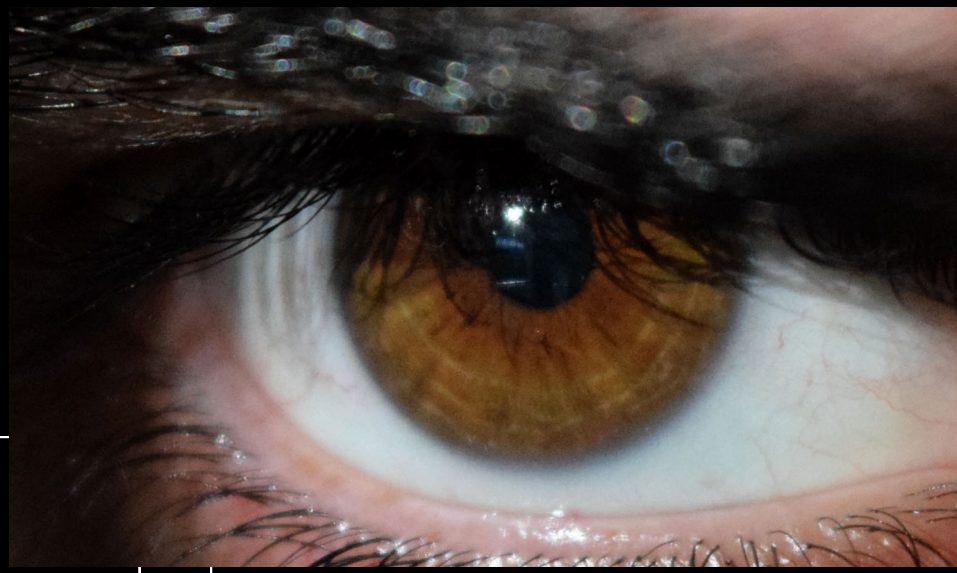
DEEP LEA

CN

CSEN3083

Full Project

https://github.com/
boga/SmartAtte



ess detecti

a binary i
er. It lear
y a face c

CSEN3083

CNN implem

train_face_model.py

.py

train_face_model.py

src > train_face_model.py > ...

```python
import argparse
from pathlib import Path

import matplotlib.pyplot as plt

from src.config import ARTIFACTS_DIR, FACE_LABELS_PATH, FACE_MODEL_PATH, FACES_DIR, IMAGE_SIZE
from src.utils import ensure_directories, save_json


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Train the face recognition CNN.")
    parser.add_argument("--epochs", type=int, default=20)
    parser.add_argument("--batch-size", type=int, default=16)
    parser.add_argument("--validation-split", type=float, default=0.2)
    parser.add_argument("--seed", type=int, default=42)
    return parser.parse_args()


def build_model(num_classes: int):
    import tensorflow as tf

    data_augmentation = tf.keras.Sequential(
        [
            tf.keras.layers.RandomFlip("horizontal"),
            tf.keras.layers.RandomRotation(0.05),
            tf.keras.layers.RandomZoom(0.1),
        ]
    )

    model = tf.keras.Sequential(
        [
            tf.keras.layers.Input(shape=(*IMAGE_SIZE, 3)),
            data_augmentation,
            tf.keras.layers.Conv2D(32, 3, activation="relu"),
            tf.keras.layers.MaxPooling2D(),
            tf.keras.layers.Conv2D(64, 3, activation="relu"),
            tf.keras.layers.MaxPooling2D(),
            tf.keras.layers.Conv2D(128, 3, activation="relu"),
```

in_liveness_model.py

train_liveness_model.py > ...

```python
def train_liveness_model(
        raise ValueError("At least two real and two fake liveness samples are required to train the

    import tensorflow as tf

    train_ds = tf.keras.utils.image_dataset_from_directory(
        LIVENESS_DIR,
        validation_split=validation_split,
        subset="training",
        seed=seed,
        image_size=IMAGE_SIZE,
        batch_size=batch_size,
        label_mode="binary",
    )
    val_ds = tf.keras.utils.image_dataset_from_directory(
        LIVENESS_DIR,
        validation_split=validation_split,
        subset="validation",
        seed=seed,
        image_size=IMAGE_SIZE,
        batch_size=batch_size,
        label_mode="binary",
    )

    class_names = list(train_ds.class_names)
    if sorted(class_names) != ["fake", "real"]:
        print(f"Warning: expected classes ['fake', 'real'], found {class_names}")

    autotune = tf.data.AUTOTUNE
    train_ds = train_ds.prefetch(buffer_size=autotune)
    val_ds = val_ds.prefetch(buffer_size=autotune)

    model = build_model()
    history = model.fit(train_ds, validation_data=val_ds, epochs=epochs)

    val_scores = model.predict(val_ds, verbose=0).flatten()
    val_labels = np.concatenate([labels.numpy().astype(int).flatten() for _, labels in val_ds], axi
    threshold, threshold_accuracy = find_best_threshold(val_scores, val_labels)
```

CSEN3083

Main Concep

reprocessin

face images
nd normalized
sed into the C
network cons
ensions and p

Hierarchical feature e

N layers lear

CSEN3083

Main Concep

ıt regulari

is used to re
g, especially
ect uses a re
stom datase

Confidence thresh

ion is not ac
use it has th

CSEN3083

THA

Ya